

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет
по Домашней работе №5
«Реализация межсервисного взаимодействия посредством очередей сообщений»

Выполнил:
Суворин Иван
БР 2.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задание

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

Выбор брокера сообщений

Выбор сделан в пользу RabbitMQ, а не Kafka, по трем причинам:

- Модель взаимодействия «сервис оповещает о событии» – это ровно то, для чего создавался RabbitMQ, в отличие от Kafka, спроектированной для потоковой обработки больших объемов данных;
- Значительно меньше сущностей для освоения: exchange/queue/routing key против topic/partition/consumer group/offset у Kafka;
- Клиентская библиотека amqp-lib – чистый JavaScript без нативной компиляции, что снижает риск проблем установки (уже возникавших в проекте с better-sqlite3).

Это подтверждается и независимыми источниками: даже после упрощений в Kafka 4.0 (отказ от ZooKeeper) RabbitMQ остается более простым для старта решением, а использование Kafka для несложных сценариев уведомлений между сервисами описывается как избыточное усложнение.

Размещение брокера

Выбран облачный хостинг CloudAMQP – это позволило сосредоточиться на коде взаимодействия, а не на настройке окружения. При переходе на Docker потребуется лишь заменить адрес подключения в .env на адрес контейнера RabbitMQ – код останется без изменений.

В процессе возникла путаница, заслуживающая отдельного упоминания – см. раздел 3.2.

Что переведено на очередь сообщений, а что – нет

К моменту этого этапа между сервисами уже существовало 8 синхронных REST-эндпоинтов (Д34/ЛР2). Переводить все на очереди было бы избыточно: 7 из них – это запросы, требующие немедленного ответа для формирования HTTP-ответа клиенту (например, «дай данные автора рецепта прямо сейчас»), и очередь сообщений для такого паттерна не подходит.

Единственный эндпоинт, для которого асинхронная модель подходит по смыслу – очистка лайков/избранного/комментариев в Social Service при удалении рецепта в Recipe Service. Это не запрос данных, а оповещение о случившемся факте, не требующее немедленного ответа – идеальный случай для очереди.

	Было	Стало
Вызов	Recipe Service делает HTTP-запрос в Social Service и ждет ответа	Recipe Service публикует сообщение и не ждет ответа
При недоступности Social Service	Все удаление рецепта отменяется целиком	Удаление проходит успешно; сообщение ждет в очереди, пока Social Service не восстановится
Связанность	Recipe Service должен знать, что Social Service жив прямо сейчас	Полная временная развязка сервисов

Технический дизайн

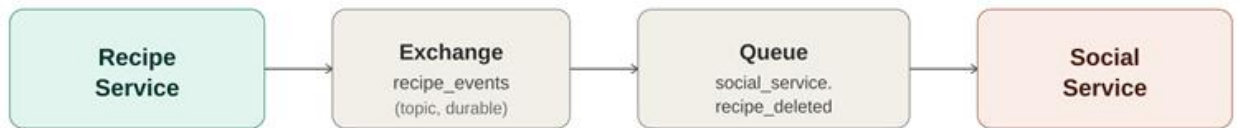


Рисунок 1 – Поток события recipe.deleted

- Exchange: recipe_events, тип topic, durable – позволяет в будущем добавлять другие события рецепта (например, recipe.published) новыми routing key без создания новых обменников;
- Routing key: recipe.deleted;
- Очередь: social_service.recipe_deleted, durable – создается и слушается Social Service, переживает перезапуск брокера вместе с необработанными сообщениями;
- Тело сообщения: { "recipeId": number }, публикуется с флагом persistent: true;
- Библиотека – amqplib (версия 0.10.7, согласованная с пакетом типов @types/amqplib 0.10.8) в обоих сервисах.

Реализация

Recipe Service – издатель события. При успешном удалении рецепта публикует сообщение вместо прежнего HTTP-вызова:

```
export async function deleteRecipe(id, currentUser) {
  // ...проверка прав, удаление ingredients/steps/recipe локально...

  // Не ждем ответа Social Service – просто оповещаем о факте
  publishRecipeDeleted(id);
}
```

Social Service – подписчик. При старте объявляет обменник и свою очередь, привязывает ее по routing key и начинает слушать:

```
await channel.assertExchange('recipe_events', 'topic', { durable: true
});

await channel.assertQueue('social_service.recipe_deleted', { durable:
true });
```

```

    await channel.bindQueue('social_service.recipe_deleted',
'recipe_events', 'recipe.deleted');

    channel.consume(Queue, async (msg) => {

        const { recipeId } = JSON.parse(msg.content.toString());

        await deleteInteractionsForRecipe(recipeId);    // та же функция, что
раньше вызывалась по HTTP

        channel.ack(msg);    // подтверждаем - сообщение можно удалить из
очереди

    });

```

Обработка ошибок при получении сообщения – `channel.nack(msg, false, true)` возвращает сообщение в очередь для повторной попытки (например, если БД была временно недоступна).

Устойчивость к обрывам соединения

Оба сервиса подключаются к RabbitMQ при старте, не блокируя запуск HTTP-сервера (если брокер временно недоступен, обычные запросы продолжают обрабатываться). При обрыве соединения предусмотрено автоматическое переподключение каждые 5 секунд с логированием попыток.

Демонстрация работы

Сценарий: создание рецепта -> отправка на модерацию -> одобрение администратором -> лайк -> комментарий -> удаление рецепта. Ниже представлен фрагмент реального журнала работы системы (оба сервиса запущены через `npm run dev`), подтверждающий, что событие проходит полный путь через облачный RabbitMQ:

```

[recipe] POST /api/recipes 200 116.377 ms - 475
[gateway] POST /api/recipes 200 120.355 ms - 475
[auth] GET /internal/users/1 200 1.816 ms - 95
[social] GET /internal/likes/count/4 200 11.063 ms - 24
[recipe] POST /api/recipes/4/submit 200 62.543 ms - 477
[gateway] POST /api/recipes/4/submit 200 69.361 ms - 477
[auth] GET /internal/users/1 200 1.394 ms - 95
[social] GET /internal/likes/count/4 200 1.730 ms - 24
[recipe] PUT /api/admin/recipes/4/approve 200 29.390 ms - 479
[gateway] PUT /api/admin/recipes/4/approve 200 32.653 ms - 479
[recipe] PUT /api/admin/recipes/4/approve 409 6.097 ms - 92

```

```
[gateway] PUT /api/admin/recipes/4/approve 409 9.762 ms - 92
[recipe] PUT /api/admin/recipes/4/approve 409 9.750 ms - 92
[gateway] PUT /api/admin/recipes/4/approve 409 14.457 ms - 92
[social] GET /internal/likes/count/4 200 0.825 ms - 24
[auth] GET /internal/users/1 200 0.975 ms - 95
[recipe] GET /internal/recipes/4 200 9.099 ms - 262
[social] POST /api/recipes/4/like 204 62.694 ms - -
[gateway] POST /api/recipes/4/like 204 67.527 ms - -
[auth] GET /internal/users/1 200 0.890 ms - 95
[social] GET /internal/likes/count/4 200 1.039 ms - 24
[recipe] GET /internal/recipes/4 200 8.544 ms - 262
[auth] GET /internal/users/1 200 2.068 ms - 95
[social] POST /api/comments/recipe/4 200 396.641 ms - 137
[gateway] POST /api/comments/recipe/4 200 404.200 ms - 137
[recipe][recipe-service] Опубликовано событие recipe.deleted (id=4)
[recipe] DELETE /api/recipes/4 204 13.398 ms - -
[gateway] DELETE /api/recipes/4 204 18.419 ms - -
[social][social-service] Получено событие recipe.deleted (id=4)
```

Необходимо обратить внимание на порядок двух последних строк: событие публикуется и клиент получает успешный ответ (204) практически одновременно, а обработка на стороне Social Service происходит отдельно, полностью асинхронно — это и есть ключевое отличие от синхронной версии ЛР2, где вся операция удаления ждала бы ответа Social Service.

Exchanges

▼ All exchanges (8)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
wjibmwtz	(AMQP default)	direct	D			
wjibmwtz	amq.direct	direct	D			
wjibmwtz	amq.fanout	fanout	D			
wjibmwtz	amq.headers	headers	D			
wjibmwtz	amq.match	headers	D			
wjibmwtz	amq.rabbitmq.trace	topic	D I			
wjibmwtz	amq.topic	topic	D			
wjibmwtz	recipe_events	topic	D	0.00/s	0.00/s	

► Add a new exchange

Рисунок 2 – Exchanges

Queues

▼ All queues (1)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Overview						Messages			Message rates			+/-
Virtual host	Name	Type	Features		State	Ready	Unacked	Total	incoming	deliver / get	ack	
wjibmwtz	social_service.recipe_deleted	classic	D	Args default wjibmwtz-max-length	running	0	0	0	0.00/s	0.00/s	0.00/s	

► Add a new queue

Рисунок 3 - Queues

ВЫВОД

Реализовано асинхронное межсервисное взаимодействие через RabbitMQ (облачный хостинг CloudAMQP) для события удаления рецепта – единственного из восьми внутренних взаимодействий, для которого паттерн «событие вместо запроса» подходит по смыслу; остальные семь осознанно оставлены синхронными REST-вызовами, так как требуют немедленного ответа. Recipe Service публикует персистентное сообщение `recipe.deleted` в topic-обменник, Social Service обрабатывает его через durable-очередь с явным подтверждением (`ack/nack`). Практическая выгода продемонстрирована и объяснена: устойчивость к временной недоступности Social Service, которой не было в синхронной версии. В процессе устранены две проблемы – согласование версий `amqp-lib/@types` и различие тарифов RabbitMQ/LavinMQ на CloudAMQP – обе задокументированы вместе с решением.